

Site isolation is a web browser security feature that groups websites into sandboxed processes by their associated origins. This technique enables the process sandbox to block cross-origin bypasses that would otherwise be exposed by exploitable vulnerabilities in the sandboxed process.

The feature was first proposed publicly by Charles Reis and others, although Microsoft was independently working on implementation in the Gazelle research browser at the same time. The approach initially failed to gain traction due to the large engineering effort required to implement it in a fully featured browser, and concerns around the real-world performance impact of potentially unbounded process use. In 2018, following the discovery of the Spectre and Meltdown vulnerabilities to the public, Google accelerated the work, culminating in a 2019 release of the feature. In 2021, Firefox also launched their own version of site isolation which they had been working on under the codename Project Fission.

Despite the security benefits of this feature, it does have limitations and tradeoffs. While it provides a baseline protection against side channel attacks such as Spectre and Meltdown, full protection against such attacks requires developers to explicitly enable certain advanced browser protections. The main tradeoff of site isolation involves the added resource consumption necessitated by the additional processes it requires. This limits its effectiveness on some classes of devices, and can be abused in some cases to enable resource exhaustion attacks.

Background

Until 2017, the predominant security architecture of major browsers adhered to the process-per-browsing-instance model. This entailed the browser comprising distinct sandboxed processes, including the browser process, GPU process, networking process, and rendering process. The rendering process would engage with other privileged services when necessary to execute elevated actions when viewing a web page.

Although this model successfully prevented problems associated with malicious JavaScript gaining access to the operating system, it lacked the capability to isolate websites from each other adequately. Despite these concerns, the adoption of a more robust model faced limited traction due to perceived issues with newer models, particularly those related to performance and memory.

In 2017, the disclosure of Spectre and Meltdown exploits, however, altered this landscape. Previously accessing arbitrary memory was complicated requiring a compromised renderer. However, with Spectre, attacks were developed that abused Javascript features to read almost all memory in the rendering process, including memory storing potentially sensitive information from previously rendered cross-origin pages. This exposed the issues of the process-per-instance security model. Consequently, a new security architecture that allowed the separation of the rendering of different web pages into entirely isolated processes was required.

History

In 2009, Reis et al. proposed the first version of the process-per-site model to isolate web pages based on the page's web origin. This was improved upon in 2009 by the Gazelle research browser, which separated specific document frames based on their web principal, a security barrier that corresponded with the specific document that was being loaded. Around the same time, work was also being done on the OP (which would later become the OP2 browser), IBOS, Tahoma and the SubOS browsers all of which proposed different paradigms to solve the issue of process separation amongst sites.

Modern implementation

In May 2013 a member of Google Chrome's Site Isolation Team announced on the chromium-dev mailing list that they would begin landing code for out-of-process i-frames

(OOPIF). This was followed by a Site Isolation Summit at BlinkOn in January 2015, which introduced the eight-engineer team and described the motivation, goals, architecture, proposed schedule, and progress made so far. The presentation also included a demo of Chrome running with an early prototype of site isolation.

In 2019, Reis, et al. of the Google Chrome project presented a paper at USENIX Security that detailed changes to their existing browser security model in response to the recent research proving that the Spectre attack could be used inside the rendering process of the browser. The paper proposed changes to the model that borrowed from Reis et al.'s work in 2009. Chrome's implementation of site isolation would use web origins as a primary differentiator of a 'site' at a process level. Additionally, the Chrome team also implemented the idea of website frames being executed out of process, a feature that had been suggested by the authors of the Gazelle web browser, as well as the OP and OP2 web browsers. This required a significant re-engineering of Chrome's process handling code, involving to more than 4000 commits from 320 contributors over a period of 5 years.

Chrome's implementation of site isolation allowed it to eliminate multiple universal cross-site scripting (uXSS) attacks. uXSS attacks allow attackers to compromise the same-origin policy, granting unrestricted access to inject and load attacker controlled javascript on other website. The Chrome team found that all 94 uXSS attacks reported between 2014 and 2018 would be rendered ineffective by the deployment of site isolation. In addition to this, the Chrome team also claimed that their implementation of site isolation would be effective at preventing variations of the Spectre and Meltdown group of timing attacks that relied on the victim address space being on the same process as the attacker process.

In March 2021, the Firefox development team announced that they would also roll out their implementation of site isolation. This feature had been in development for multiple months under the codename Project Fission. Firefox's implementation fixed a few of the flaws that had been found in Chrome's implementation namely the fact that similar web pages were still

vulnerable to uXSS attacks. The project also required a rewrite of the process handling code in Firefox.

Reception

Before 2019, site isolation had only been implemented by research browsers. Site isolation was considered to be resource intensive due to an increase in the amount of memory space taken up by the processes. This performance overhead was reflected in real world implementations as well. Chrome's implementation of site isolation on average took one to two cores more than the same without site isolation. Additionally, engineers working on the site isolation project observed a 10 to 13 percent increase in memory usage when site isolation was used.

Chrome was the industry's first major web browser to adopt site isolation as a defense against uXSS and transient execution attacks. To do this, they overcame multiple performance and compatibility hurdles, and in doing so, they kickstarted an industry-wide effort to improve browser security. However, despite this, certain aspects of Spectre's defenses have been found lacking. In particular, site isolation's ability to defend against timing attacks has been found to be incomplete. In 2021, Agarwal et al. were able to develop an exploit called Spook.js that was able to break Chrome's Spectre defenses and exfiltrate data across web page in different origins. In the same year, researchers at Microsoft, were able to leverage site isolation to perform a variety of timing attacks that allowed them to leak cross-origin information by careful manipulation of the inter-process communication protocols employed by site isolation.

In 2023, researchers at Ruhr University Bochum showed that they were able to leverage the process architecture required by site isolation to exhaust system resources and also perform advanced attacks like DNS poisoning.

References

Citations

=== Sources ===